Lab 5 — Pull Requests, Code Review, and the Cloud Coding Agent

After Module 5 — Code Reviews and Pull Requests

You will add the Category field — Version 3 — without writing the code yourself. Confirm your TodoApp repository is pushed and main is up to date.

Before you start: the starter repository already has a branch ruleset on main requiring one approval and an automatic Copilot review on draft pull requests.

Step 1 — Move the project to Version 3

Open .github/copilot-instructions.md. Update the version list:

```
- Version 1 (complete): in-memory storage, no database
- Version 2 (complete): SQLite via SQLAlchemy with Alembic migrations
- Version 3 (current): Category field added to todos
```

Under Do NOT, delete the line forbidding the Category field.

Then create .github/instructions/code-review.instructions.md:

```
---
applyTo: "**"
---

# Code review focus for TodoApp

Project: single-user Todo app. Backend Python 3.12 + FastAPI +
SQLAlchemy + Alembic. Frontend React with Vite, JavaScript.

Focus on:
 - Layer violations. A router must not import the repository. A
   service must not import FastAPI. Data access appears only in the
   repository layer. Flag every breach.
 - Migrations. Every model change must have a matching Alembic
   migration. Flag a model change with no migration. Flag a migration
   with no downgrade.
 - Status codes. 201 on create, 204 on delete, 404 for an unknown id,
   422 for invalid input. Flag anything returning 200 for a create.
 - Datetimes. DateCreated must be timezone-aware UTC. Flag any naive
   datetime.
 - Field naming. Id, Title, Description, DateCreated, IsCompleted,
```

```
  Category. Flag any renaming or casing drift between backend and
  frontend.
- Frontend API calls. All calls go through src/api/todoApi.js. Flag
  any fetch call made directly from a component.

Do NOT comment on:
- Formatting, indentation, or line length.
- Missing type hints on private helper functions.
- Missing tests. Tests are covered separately.
- Suggestions to add authentication, error middleware, or logging
  frameworks. They are deliberately out of scope.
```

```
git add .
git commit -m "Move to Version 3 and add code review instructions"
git push
```

Step 2 — Create the issue

On github.com, open your TodoApp repository. Click Issues, then New issue.

Title:

```
Add Category field to todos (Version 3)
```

Body:

```
## Goal
Add a Category field to every todo, end to end.


## Requirements


Data model
- New field `Category`, a string, maximum 50 characters, optional.
- Existing todos with no category are valid. Do not backfill a default.


Database
- Add an Alembic migration for the new column.
- The migration must have a working downgrade.
- Do not delete or recreate the todos table.
```

Backend
- Category is accepted on create and on update.
- Category is returned on every read.
- `GET /todos` accepts an optional `category` query parameter that filters by exact match. Filtering happens in the SQLAlchemy query, not in Python.
- Existing filter and sort query parameters must keep working, and must combine with the category filter.

Frontend
- A text input for Category on the add-todo form.
- Category is displayed on each todo in the list.
- A dropdown that filters by category. Its options are built from the categories present in the current todo list, plus an "All" option.

Constraints
- Follow the layer rules in .github/copilot-instructions.md.
- Do not add authentication, a UI library, or TypeScript.
- Do not change any existing field name.
- Do not modify docs/PRD.md or docs/TRD.md.

Acceptance
- Application runs. Existing todos still load after the migration.
- A todo can be created with and without a category.
- Filtering by category works and combines with the completed filter.

Click Submit new issue. Notice how specific this is. The agent will receive nothing else.

Step 3 — Assign the issue to Copilot

In the issue's right-hand sidebar, click Assignees. Select Copilot.

The agent adds an eyes reaction within a few seconds and begins work. It will open a draft pull request shortly after.

If Copilot does not appear in the assignee list, your plan or organisation policy does not include the coding agent. Build the feature locally with agent mode instead, then open a normal pull request — the rest of the lab is unchanged.

Step 4 — Watch it work

Open the draft pull request the agent created. Expand its progress log.

Read what it did: which files it read, what it decided, which commands it ran, and where it corrected itself.

While it works, read the issue you wrote again and predict where it will go wrong.

Step 5 — Read the self-summary and generate the PR summary

Read the agent's own summary at the top of the pull request. It lists what it changed and, if self-review ran, which issues it already fixed before you arrived. That tells you which parts of the diff have already had a review pass.

Now click the Copilot icon in the pull request description box and choose Summary.

Read what it generates, then edit it by hand to add two things Copilot cannot know: why this change is being made, and a link to the issue written as Closes #1 so the issue closes on merge.

Step 6 — Request a Copilot review

- In the Reviewers menu, select Copilot.
- Wait. A small pull request usually takes under 30 seconds.
- Read every comment.

For each comment, decide one of three things and act on it. Correct and worth fixing: click Apply suggestion, or batch several and commit them together. Correct but not now: reply explaining why, and resolve. Wrong: reply saying why it is wrong.

Note that Copilot left a Comment review, not an Approve or Request changes. It never blocks a merge.

Step 7 — Review the diff yourself

Copilot will not catch everything. Check these two by hand:

- Open the Alembic migration. Does the downgrade function actually drop the column, and does down_revision point at your Version 2 migration?
- Does the category filter combine correctly with the completed filter, or does one overwrite the other?

Step 8 — Iterate with the agent

Do not fix the problems yourself. Leave a review comment on the pull request:

```
@copilot The category filter and the completed filter do not combine -
passing both returns only the category matches. Fix the repository query
so all filters apply together, and confirm which file you changed.
```

The agent reacts with an eyes emoji and pushes a commit to the same branch. Batch your feedback where you can rather than sending one comment at a time.

Step 9 — Test locally and merge

Check the branch out:

```
git fetch origin
git checkout <the-agent-branch-name>
cd backend
pip install -r requirements.txt
alembic upgrade head
uvicorn main:app --reload
```

Confirm the todos you created in Lab 4 are still present after the migration. Add a todo with a category, one without. Filter by category. Combine it with the completed filter.

Mark the pull request Ready for review, then merge it. Confirm the issue closed automatically. If it did not, your Closes #1 line was missing or misspelled.

```
git checkout main
git pull
```

What you should have at the end of this lab

Version 3 is merged. Every todo has an optional category, filterable end to end.

You wrote an issue and reviewed a diff. You did not write the feature.